Step 4 — A Test Suite That Catches a Bad Change Before It Ships (final) — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 4: the API tests prove the **service** works. They say nothing about whether the **articles** are still good — the team is faked there on purpose. This step adds Doc13's eval suite: 10 fixed topics, each with a check written **before** anything runs, a judge for the parts code can't check, and a regression check that makes a prompt worse on purpose and proves the score drops. **If not:** a prompt edit that quietly drops the `[source]` tags would pass every test and ship.

After this step the project is complete. Files not shown here don't change.

Project 4's test files stay exactly as they were — plain OK/WRONG scripts; this step only **adds** tests, which use pytest (Doc13's style).

Back to this step · Hint 1 · Hint 2 · Solution

Solution — part 1: the suite, one job per file

Story — `evals/tasks.py`: the fixed test set, written **before** anything runs (Doc13). Source tags are exact, so they get a free code check; "says how long panels take to pay back" needs the judge; unknown topics must produce **no** article. (Also create an empty `evals/__init__.py`.) **If not:** you'd write the rules after reading the output, and they'd describe whatever the team wrote.

```
# src/evals/tasks.py
# The fixed eval set, written BEFORE running anything: 10 topics, each with
# an exact-text check, judge rules, or "expect no article".
TITLE_RULE = "starts with a short title on its own line"
SOURCE_RULE = "every fact in the article ends with a [source] tag"

TASKS = [
    # exact-check tasks: one looked-up source tag that must appear
    {"id": "ebike_range", "topic": "electric bikes",
     "expected": "[battery-review]"},
    {"id": "ebike_rules", "topic": "EU rules for electric bikes",
     "expected": "[eu-rules]"},
    {"id": "remote_commute", "topic": "remote work",
     "expected": "[commute-study]"},
    {"id": "solar_lifetime", "topic": "solar panels",
     "expected": "[solar-guide]"},
    # judge tasks: rules about meaning, checked by judge.py
    {"id": "ebike_popular", "topic": "why electric bikes are popular",
     "rules": ["says how far an e-bike goes on one charge",
               SOURCE_RULE]},
    {"id": "remote_downside", "topic": "the pros and cons of remote work",
     "rules": ["names at least one downside of remote work",
               SOURCE_RULE]},
    {"id": "solar_payback", "topic": "are solar panels worth it",
     "rules": ["says how long panels take to pay for themselves",
               SOURCE_RULE]},
    {"id": "solar_shape", "topic": "solar panels for a family home",
     "rules": [TITLE_RULE, SOURCE_RULE]},
    # no-article tasks: nothing is known, so the team must stop
    {"id": "unknown_mining", "topic": "deep sea mining",
     "expect_no_article": True},
    {"id": "unknown_quantum", "topic": "quantum computing",
```

```
     "expect_no_article": True},
]
```

Story — `evals/judge.py`: Doc13's judge, unchanged except two lines: the model name comes from `models/llm.py`, and the OpenAI client is made inside `call_model()`. Typed result, versioned prompt, `temperature=0`. **If not:** creating the client at import time would crash before `load_config()` had read `.env`.

```
# src/evals/judge.py
# The LLM judge. score_with_judge() scores an article against written rules
# and reads PASS/FAIL from the first line; the client is made inside
# call_model(), not at import time.
from dataclasses import dataclass

from openai import OpenAI

from src.models.llm import MODEL_NAME

JUDGE_PROMPT_VERSION = "v1"

@dataclass
class JudgeResult:
    verdict: str
    reason: str

def call_model(prompt: str) -> str:
    client = OpenAI()
    response = client.chat.completions.create(
        model=MODEL_NAME,
        temperature=0,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content

def build_judge_prompt(task_input: str, output: str, rules: list) -> str:
    rules_text = ""
    for rule in rules:
        rules_text = rules_text + "- " + rule + "\n"
    return (
        "Task: " + task_input + "\n"
        "Answer to check: " + output + "\n"
        "Rules the answer must follow:\n" + rules_text +
        "Reply with PASS or FAIL on the first line, "
        "then one sentence explaining why."
    )
```

```
def score_with_judge(task_input: str, output: str, rules: list):
    reply = call_model(build_judge_prompt(task_input, output, rules))
    lines = reply.strip().split("\n", 1)
    verdict = lines[0].strip().upper()
    if len(lines) > 1:
        reason = lines[1].strip()
    else:
        reason = ""
    return JudgeResult(verdict=verdict, reason=reason)
```

Story — `evals/run_eval.py`: Doc13's runner, pointed at `run_pipeline()` — the same door the API uses, so the suite tests exactly what ships. The cheapest check that works for each task; a reason for every failure; a saved `report.json`. **If not:** a run's result would live only in your terminal, with no reason for a failure.

```
# src/evals/run_eval.py
# Runs the eval suite against run_pipeline() (the same door the API uses),
# checks each task the cheapest way that works, prints the score and saves
# report.json.
import json

from src.config import load_config
from src.evals.judge import JUDGE_PROMPT_VERSION, score_with_judge
from src.evals.tasks import TASKS
from src.pipeline import run_pipeline

def run_one_task(task: dict) -> dict:
    outcome = run_pipeline(task["topic"])
    article = outcome["article"]
    if "expect_no_article" in task:
        passed = article == ""
        reason = "expected no article, got " + str(len(article)) + " chars"
    elif "expected" in task:
        passed = task["expected"].lower() in article.lower()
        reason = "looked for: " + task["expected"]
    else:
        judged = score_with_judge(task["topic"], article, task["rules"])
        passed = judged.verdict == "PASS"
        reason = judged.reason
    return {"task_id": task["id"], "passed": passed, "reason": reason}

def run_eval_suite() -> dict:
    results = []
    passed_by_id = {}
    for task in TASKS:
        result = run_one_task(task)
```

```
        results.append(result)
        passed_by_id[result["task_id"]] = result["passed"]

    passed_count = 0
    for result in results:
        if result["passed"]:
            passed_count = passed_count + 1
    overall_score = passed_count / len(results)

    report = {
        "judge_prompt_version": JUDGE_PROMPT_VERSION,
        "overall_score": overall_score,
        "passed": passed_count,
        "total": len(results),
        "results": results,
    }
    with open("report.json", "w") as f:
        json.dump(report, f, indent=2)

    print(f"Passed {passed_count}/{len(results)} "
          f"(score: {overall_score:.2f})")
    for result in results:
        if not result["passed"]:
            print("  FAILED " + result["task_id"] + ": " + result["reason"])

    return passed_by_id

if __name__ == "__main__":
    load_config()
    run_eval_suite()
```

Run it: `python -m src.evals.run_eval`

```
Passed 10/10 (score: 1.00)
```

Part 2 — prove the judge, and prove the suite catches a regression

Story — `tests/integration/test_judge.py`: Doc13's calibration set, for this suite's rules: the judge must PASS a titled, tagged article and FAIL an off-topic one and one with no tags. **If not:** a judge that approves everything would make every run look perfect.

```
# tests/integration/test_judge.py
# Calls the real judge — run it whenever judge.py's prompt changes.
# Run from the repo root: pytest tests/integration/test_judge.py -v
from src.config import load_config
```

```
from src.evals.judge import score_with_judge
from src.evals.tasks import SOURCE_RULE, TITLE_RULE

load_config()

TOPIC = "electric bikes"
RULES = [TITLE_RULE, SOURCE_RULE]

# answers where YOU already know the right verdict — a judge that approves
# everything is caught here, before it grades anything
CALIBRATION_SET = [
    {"output": "E-Bikes Explained\n\nA battery gives 40 to 100 km "
               "per charge. [battery-review]",
     "expected_verdict": "PASS"},
    {"output": "I like pizza.",
     "expected_verdict": "FAIL"},
    {"output": "E-Bikes Explained\n\nA battery gives 40 to 100 km "
               "per charge.",
     "expected_verdict": "FAIL"},
]

def test_judge_calibration():
    for case in CALIBRATION_SET:
        result = score_with_judge(TOPIC, case["output"], RULES)
        assert result.verdict == case["expected_verdict"], result.reason
```

Story — `evals/regression_check.py` : Doc13's pass bar and per-task list, run against the real suite. The sabotage is in **Research's** prompt: without "each ending with its [source] tag", the notes lose their sources, and nothing later in the team can put them back. **If not:** "the suite would catch a bad change" would stay a belief, never a result.

```
# src/evals/regression_check.py
# Proves the eval suite catches a bad change: run it, pause while you break
# Research's prompt on purpose, run it again, and report the score drop.
from src.config import load_config
from src.evals.run_eval import run_eval_suite

# decided BEFORE sabotaging, so the result can't be explained away after
PASS_BAR = 0.8

SABOTAGE = (
    'In src/prompts/research.txt, delete the words\n'
    '", each ending with its [source] tag". Save, then press Enter...'
)

def score(passed_by_id: dict) -> float:
```

```
    passed = 0
    for task_id in passed_by_id:
        if passed_by_id[task_id]:
            passed = passed + 1
    return passed / len(passed_by_id)

def find_newly_failed(baseline: dict, after: dict) -> list:
    newly_failed = []
    for task_id in baseline:
        if baseline[task_id] and not after[task_id]:
            newly_failed.append(task_id)
    return newly_failed

def run_regression_check() -> None:
    print("--- baseline run ---")
    baseline = run_eval_suite()
    input(SABOTAGE)
    print("--- after sabotage ---")
    after = run_eval_suite()

    newly_failed = find_newly_failed(baseline, after)
    print(f"score: {score(baseline):.2f} -> {score(after):.2f}")
    if score(baseline) >= PASS_BAR and score(after) < PASS_BAR:
        print("PASS: the suite caught it. Newly failing:")
        for task_id in newly_failed:
            print("  " + task_id)
    else:
        print("FAIL: the suite did NOT catch it. Tighten the rules.")

if __name__ == "__main__":
    load_config()
    run_regression_check()
```

Run everything (from the repo root):

```
python -m tests.unit.test_config       # free — Project 4's own tests
python -m tests.unit.test_lookup
python -m tests.unit.test_supervisor
pytest tests/api -v                    # free — the API, team faked
python -m tests.integration.test_live_agents      # real API calls
pytest tests/integration/test_judge.py -v         # real API calls
python -m src.evals.run_eval
python -m src.evals.regression_check
```

Expected output of the regression check (the second run's `FAILED` lines shortened):

```
--- baseline run ---
Passed 10/10 (score: 1.00)
In src/prompts/research.txt, delete the words
", each ending with its [source] tag". Save, then press Enter...
--- after sabotage ---
Passed 2/10 (score: 0.20)
  FAILED ebike_range: looked for: [battery-review]
  ...
score: 1.00 -> 0.20
PASS: the suite caught it. Newly failing:
  ebike_range
  ebike_rules
  remote_commute
  solar_lifetime
  ebike_popular
  remote_downside
  solar_payback
  solar_shape
```

The 2 unknown-topic tasks still pass — the sabotage didn't touch them. Your numbers may differ a little with a real model; if the score does **not** drop below the bar, a rule is too loose — tighten it (Doc13's `regression_catch`). **Put the words back** in `research.txt` when you're done.